

### Question description

Teddy is visiting the country wonderland. wonderland has  $n$  cities and  $m$  bi-directional roads. There are  $k$  types of tokens. Token  $i$  costs  $c_i$ . The costs of the tokens are such that for all  $2 \leq i \leq k$ ,  $c_i \geq 2c_{i-1}$ . For each road, you need to have a particular set of tokens, if you want to travel it. Note that you don't have to give the tokens, you just need to show them. Thus, one token can be used at any number of roads, where it is required. Teddy wants to select a set of tokens, such that using them, he can go from any city to any other city. You have to help him minimize the total cost of tokens he buys.

### Constraints

$$1 \leq n \leq 10^5$$

$$1 \leq m \leq 10^5$$

No road connects a city to the same city. However, there can be multiple roads between two cities.

$$1 \leq c_i \leq 10^{18}$$

For all  $2 \leq i \leq k$ ,  $c_i \geq 2c_{i-1}$

### Input:

- The first line contains three space separated integers,  $n$   $m$  and  $k$ .
- The second line contains  $k$  space separated integers, where the  $i^{\text{th}}$  integer denotes the price of  $i^{\text{th}}$  token, i.e.  $c_i$ .
- $i^{\text{th}}$  of the next  $m$  lines contains three integers  $u_i, v_i, l_i$ , where  $l_i$  is the number of tokens required by the  $i^{\text{th}}$  road, followed by  $l_i$  indices denoting the tokens required. This road connects cities  $u_i$  and  $v_i$

### Output:

- Print one integer containing the minimum cost of tokens Teddy has to buy, such that he can travel from any city to any other city. If it is impossible to choose such a set of tokens, print  $-1$ .

▼ Logical Test Cases

Test Case 1

INPUT (STDIN)

3 3 4  
1 2 5 10  
1 2 2 1 2  
1 3 1 3  
2 3 1 4

EXPECTED OUTPUT

8

Test Case 2

INPUT (STDIN)

3 3 4  
11 21 15 10  
1 21 2 11 2  
1 13 1 3  
2 31 1 4

EXPECTED OUTPUT

57

▼ Mandatory Test Cases

Test Case 1

KEYWORD

for(i=1;i<=n;++i)

▼ Complexity Test Cases

Test Case 1

CYCLOMATIC COMPLEXITY

2

Test Case 2

TOKEN COUNT

480

Test Case 3

NLOC

60

Graph1.c +

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 #define MAXN 100005
5
6 typedef struct {
7     int u, v;
8     int l;
9     int *tokens;
10 } Road;
11
12 int parent[MAXN];
13
14 int find(int i) {
15     if (parent[i] == i)
16         return i;
17     return parent[i] = find(parent[i]);
18 }
19
20 int is_connected(int n, Road *roads, int m, int *banned_token) {
21     int i;
22     for(i=0;i<MAXN;++i) {
23         parent[i] = i;
24     }
25     int comps = n;
```

```
26     for (i = 0; i < m; i++) {
27         int valid = 1;
28         for (int j = 0; j < roads[i].l; j++) {
29             if (banned_token[roads[i].tokens[j]]) {
30                 valid = 0;
31                 break;
32             }
33         }
34         if (valid) {
35             int root_u = find(roads[i].u);
36             int root_v = find(roads[i].v);
37             if (root_u != root_v) {
38                 parent[root_u] = root_v;
39                 comps--;
40             }
41         }
42     }
```

```
43     return comps == 1;
44 }
45
```

main | CH.SC.U4CSE25225 | Graph1.c

Ln 90, Col 1 Spaces: 4 UTF-8 CRLF C

```
47     int n, m, k;
48     if (scanf("%d %d %d", &n, &m, &k) != 3) return 0;
49
50     long long *c = (long long*)malloc((k + 1) * sizeof(long long));
51     for (int i = 1; i <= k; i++) {
52         if (scanf("%lld", &c[i]) != 1) {
53             c[i] = 0;
54         }
55     }
56
57     Road *roads = (Road*)malloc(m * sizeof(Road));
58     for (int i = 0; i < m; i++) {
```

```

59     if (scanf("%d %d %d", &roads[i].u, &roads[i].v, &roads[i].l) != 3) {
60         roads[i].u = 0;
61         roads[i].v = 0;
62         roads[i].l = 0;
63     }
64     roads[i].tokens = (int*)malloc(roads[i].l * sizeof(int));
65     for (int j = 0; j < roads[i].l; j++) {
66         if (scanf("%d", &roads[i].tokens[j]) != 1) {
67             roads[i].tokens[j] = 0;
68         }
69     }
70 }
71
72 int *banned = (int*)calloc(MAXN, sizeof(int));
73 if (!is_connected(n, roads, m, banned)) {
74     printf("-1\n");

```

```
75     return 0;
76 }
77
78 long long total_cost = 0;
79 for (int i = k; i >= 1; i--) {
80     banned[i] = 1;
81     if (!is_connected(n, roads, m, banned)) {
82         banned[i] = 0;
83         total_cost += c[i];
84     }
85 }
86
87 printf("%lld\n", total_cost);
88 return 0;
89}
90
```

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t1.txt

3 3 4

1 2 5 10

1 2 2 1 2

1 3 1 3

2 3 1 4

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Graph1.exe

8

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t2.txt

3 3 2

4 8

1 2 1 1

2 3 1 2

1 3 1 2

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Graph1.exe

8

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>

|    |         |       |                      |         |              |
|----|---------|-------|----------------------|---------|--------------|
| DS | Session | Graph | Question Information | Level 1 | Challenge 82 |
|----|---------|-------|----------------------|---------|--------------|

**Question description**

Udaya family is going to order a large pizza and eat it together. A total of  $n$  family members will join the order, and there are  $m$  possible toppings. The pizza may have any number of toppings.

Each family member gives two wishes concerning the toppings of the pizza. The wishes are of the form "topping  $x$  is good/bad". Your task is to choose the toppings so that at least one wish from everybody becomes true (a good topping is included in the pizza or a bad topping is not included).

**Input**

The first input line has two integers  $n$  and  $m$ : the number of family members and toppings. The toppings are numbered  $1, 2, \dots, m$ .

After this, there are  $n$  lines describing the wishes. Each line has two wishes of the form "+  $x$ " (topping  $x$  is good) or "-  $x$ " (topping  $x$  is bad).

**Output**

Print a line with  $m$  symbols: for each topping "+" if it is included and "-" if it is not included. You can print any valid solution.

If there are no valid solutions, print "IMPOSSIBLE".

**Constraints**

- $1 \leq n, m \leq 10^5$
- $1 \leq x \leq m$

▼ Logical Test Cases

Test Case 1

INPUT (STDIN)

3 5  
+ 1 + 2  
- 1 + 3  
+ 4 - 2

EXPECTED OUTPUT

- + + + -

Test Case 2

INPUT (STDIN)

3 5  
+ 2 + 2  
- 3 + 3  
+ 5 - 2

EXPECTED OUTPUT

- + - - +

▼ Mandatory Test Cases

Test Case 1

KEYWORD

void link(int i,int j)

▼ Complexity Test Cases

Test Case 1

CYCLOMATIC COMPLEXITY

1

Test Case 2

TOKEN COUNT

600

Test Case 3

NLOC

71

Graph2.c +

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 #define MAXV 200005
5
6 int head[MAXV], to[MAXV * 4], nxt[MAXV * 4], edge_cnt;
7 int rhead[MAXV], rto[MAXV * 4], rnxt[MAXV * 4], redge_cnt;
8 int visited[MAXV];
9 int order[MAXV], order_cnt;
10 int scc[MAXV], scc_cnt;
11
12 void link(int i, int j) {
13     to[++edge_cnt] = j;
14     nxt[edge_cnt] = head[i];
15     head[i] = edge_cnt;
16
17     rto[++redge_cnt] = i;
18     rnxt[redge_cnt] = rhead[j];
19     rhead[j] = redge_cnt;
20 }
21
22 void dfs1(int u) {
23     visited[u] = 1;
24     for (int e = head[u]; e; e = nxt[e]) {
25         int v = to[e];
```

```

26     if (!visited[v]) dfs1(v);
27 }
28 order[++order_cnt] = u;
29}
30
31void dfs2(int u, int id) {
32     scc[u] = id;
33     for (int e = rhead[u]; e; e = rnxt[e]) {
34         int v = rto[e];
35         if (!scc[v]) dfs2(v, id);
36     }
37}
38
39int node(int x, int sign) {
40     if (sign == 1) return 2 * x;
41     else return 2 * x + 1;
42}

```

43

```
44int neg_node(int u) {
```

```
45    return u ^ 1;
```

main | CH.SC.U4CSE25225 | Graph2.c

Ln 90, Col 1 Spaces: 4 UTF-8 CRLF C

47

```
48int main() {
```

```
49    int n, m;
```

```
50    if (scanf("%d %d", &n, &m) != 2) return 0;
```

51

```
52    for (int i = 0; i < n; i++) {
```

```
53        char s1, s2;
```

```
54        int u, v;
```

```
55        scanf(" %c %d %c %d", &s1, &u, &s2, &v);
```

```
56        int n1 = node(u, s1 == '+' ? 1 : 0);
```

```
57        int n2 = node(v, s2 == '+' ? 1 : 0);
```

```
58        link(neg_node(n1), n2);
```

```

59     link(neg_node(n2), n1);
60 }
61
62 for (int i = 2; i <= 2 * m + 1; i++) {
63     if (!visited[i]) dfs1(i);
64 }
65
66 for (int i = order_cnt; i >= 1; i--) {
67     int u = order[i];
68     if (!scc[u]) {
69         dfs2(u, ++scc_cnt);
70     }
71 }
72
73 for (int i = 1; i <= m; i++) {
74     if (scc[node(i, 1)] == scc[node(i, 0)]) {

```

```
75         printf("IMPOSSIBLE\n");
76         return 0;
77     }
78 }
79
80 for (int i = 1; i <= m; i++) {
81     if (scc[node(i, 1)] > scc[node(i, 0)]) {
82         printf("+ ");
83     } else {
84         printf("- ");
85     }
86 }
87 printf("\n");
88 return 0;
89}
90
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t1.txt
```

```
3 5
```

```
+ 1 + 2
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Graph2.exe
```

```
+ + + + +
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t2.txt
```

```
3 5
```

```
+ 2 + 2
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Graph2.exe
```

```
+ + + + +
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>
```

|    |         |       |                      |         |              |
|----|---------|-------|----------------------|---------|--------------|
| DS | Session | Graph | Question Information | Level 1 | Challenge 83 |
|----|---------|-------|----------------------|---------|--------------|

Question description

A game consists of  $n$  rooms and  $m$  teleporters. At the beginning of each day, you start in room 1 and you have to reach room  $n$ .

You can use each teleporter at most once during the game. How many days can you play if you choose your routes optimally?

Constraints

- $2 \leq n \leq 500$
- $1 \leq m \leq 1000$
- $1 \leq a, b \leq n$

Input

The first input line has two integers  $n$  and  $m$ : the number of rooms and teleporters. The rooms are numbered  $1, 2, \dots, n$ .  
After this, there are  $m$  lines describing the teleporters. Each line has two integers  $a$  and  $b$ : there is a teleporter from the room  $a$  to room  $b$ .  
There are no two teleporters whose starting and ending rooms are the same.

Output

First print an integer  $k$ : the maximum number of days you can play the game. Then, print  $k$  route descriptions according to the example. You can print any valid solution.

▼ Logical Test Cases

| Test Case 1                                                                             | Test Case 2 |
|-----------------------------------------------------------------------------------------|-------------|
| CH.SC.U4CSE25225 - M ASHWIN   Department of Computer Science & Engineering (CSE)   2026 |             |

INPUT (STDIN)

6 7  
1 2  
1 3  
2 6  
3 4  
3 5  
4 6  
5 6

EXPECTED OUTPUT

2  
4  
1 3 5 6  
3  
1 2 6

INPUT (STDIN)

6 7  
1 2  
2 3  
3 6  
4 4  
4 5  
5 6  
5 6

EXPECTED OUTPUT

1  
4  
1 2 3 6

▼ Mandatory Test Cases

Test Case 1

KEYWORD

int bfs(int n,int s,int t)

▼ Complexity Test Cases

Test Case 1

CYCLOMATIC COMPLEXITY

7

Test Case 2

TOKEN COUNT

920

Test Case 3

NLOC

110

Graph3.c +

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <string.h>
4
5 #define MAXN 505
6 #define MAXM 2005
7
8 int head[MAXN], to[MAXM * 2], cap[MAXM * 2], flow[MAXM * 2], nxt[MAXM * 2], edge_cnt;
9 int parent_edge[MAXN];
10 int parent_node[MAXN];
11
12 void add_edge(int u, int v) {
13     to[edge_cnt] = v;
14     cap[edge_cnt] = 1;
15     flow[edge_cnt] = 0;
16     nxt[edge_cnt] = head[u];
17     head[u] = edge_cnt++;
18
19     to[edge_cnt] = u;
20     cap[edge_cnt] = 0;
21     flow[edge_cnt] = 0;
22     nxt[edge_cnt] = head[v];
23     head[v] = edge_cnt++;
24 }
25
```

```

26 int bfs(int n, int s, int t) {
27     int q[MAXN], front = 0, rear = 0;
28     int visited[MAXN];
29     memset(visited, 0, sizeof(visited));
30     memset(parent_edge, -1, sizeof(parent_edge));
31
32     q[rear++] = s;
33     visited[s] = 1;
34
35     while (front < rear) {
36         int u = q[front++];
37         if (u == t) return 1;
38
39         for (int e = head[u]; e != -1; e = nxt[e]) {
40             int v = to[e];
41             if (!visited[v] && cap[e] - flow[e] > 0) {
42                 visited[v] = 1;

```

```

43         parent_edge[v] = e;
44         parent_node[v] = u;
45         q[rear++] = v;
46     }
47 }
48 }
49 return 0;
50}

```

```
51
```

```
52int max_flow(int n, int s, int t) {
```

```
53     int total_flow = 0;
```

```
54     while (bfs(n, s, t)) {
```

```
55         for (int v = t; v != s; v = parent_node[v]) {
```

```
56             int e = parent_edge[v];
```

```
57             flow[e] += 1;
```

```
58             flow[e ^ 1] -= 1;
```

```
59         }
```

```
60     total_flow++;
61 }
62 return total_flow;
63}
64
65int path_nodes[MAXN];
66
67int main() {
68     int n, m;
69     if (scanf("%d %d", &n, &m) != 2) return 0;
70
71     memset(head, -1, sizeof(head));
72     edge_cnt = 0;
73
74     for (int i = 0; i < m; i++) {
75         int u, v;
```

```
76     scanf("%d %d", &u, &v);
77     add_edge(u, v);
78 }
79
80 int k = max_flow(n, 1, n);
81 printf("%d\n", k);
82
83 for (int i = 0; i < k; i++) {
84     int curr = 1;
85     int path_len = 0;
86     path_nodes[path_len++] = curr;
87
88     while (curr != n) {
89         for (int e = head[curr]; e != -1; e = nxt[e]) {
90             if (flow[e] > 0) {
91                 flow[e] = 0;
```

```

92         curr = to[e];
93         path_nodes[path_len++] = curr;
94         break;
95     }
96 }
97 }
98
99 printf("%d\n", path_len);
100 for (int j = 0; j < path_len; j++) {
101     printf("%d%c", path_nodes[j], j == path_len - 1 ? '\n' : ' ');
102 }
103 }
104
105 return 0;
106 }
107

```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t1.txt
```

```
6 7
```

```
1 2
```

```
1 3
```

```
2 6
```

```
3 4
```

```
3 5
```

```
4 6
```

```
5 6
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Graph3.exe
```

```
2
```

```
4
```

```
1 3 5 6
```

```
3
```

```
1 2 6
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t2.txt
```

```
6 7
```

```
1 2
```

```
2 3
```

```
3 6
```

```
4 4
```

```
4 5
```

```
5 6
```

```
5 6
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Graph3.exe
```

```
1
```

```
4
```

```
1 2 3 6
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>
```



**Question description**

Byteland has  $n$  cities and  $m$  roads between them. The goal is to construct new roads so that there is a route between any two cities. Your task is to find out the minimum number of roads required, and also determine which roads should be built.

**Constraints**

$$1 \leq n \leq 10^5$$

$$1 \leq m \leq 2 \cdot 10^5$$

$$1 \leq a, b \leq n$$

**Input**

The first input line has two integers  $n$  and  $m$ : the number of cities and roads. The cities are numbered  $1, 2, \dots, n$ .

After that, there are  $m$  lines describing the roads. Each line has two integers  $a$  and  $b$ : there is a road between those cities.

A road always connects two different cities, and there is at most one road between any two cities.

**Output**

First print an integer  $k$ : the number of required roads.

Then, print  $k$  lines that describe the new roads. You can print any valid solution.

## ✓ Logical Test Cases

| Test Case 1       |
|-------------------|
| INPUT (STDIN)     |
| 4 2<br>1 2<br>3 4 |
| EXPECTED OUTPUT   |
| 1<br>2 4          |

| Test Case 2       |
|-------------------|
| INPUT (STDIN)     |
| 4 2<br>2 3<br>2 4 |
| EXPECTED OUTPUT   |
| 1<br>1 4          |

## ✓ Mandatory Test Cases

| Test Case 1             |
|-------------------------|
| KEYWORD                 |
| <code>while(m--)</code> |

## ✓ Complexity Test Cases

| Test Case 1           |
|-----------------------|
| CYCLOMATIC COMPLEXITY |
| 5                     |

| Test Case 2 |
|-------------|
| TOKEN COUNT |
| 475         |

| Test Case 3 |
|-------------|
| NLOC        |
| 50          |

Graph4.c +

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 #define MAXN 100005
5
6 int parent[MAXN];
7
8 int find(int i) {
9     if (parent[i] == i)
10         return i;
11     return parent[i] = find(parent[i]);
12 }
13
14 int main() {
15     int n, m;
16     if (scanf("%d %d", &n, &m) != 2) return 0;
17
18     for (int i = 1; i <= n; i++) {
19         parent[i] = i;
20     }
21
22     while (m--) {
23         int u, v;
24         scanf("%d %d", &u, &v);
25         int fu = find(u),
```

```
26     int rv = find(v);
27     if (ru != rv) {
28         parent[ru] = rv;
29     }
30 }
31
32 int reps[MAXN];
33 int rep_cnt = 0;
34 for (int i = 1; i <= n; i++) {
35     if (find(i) == i) {
36         reps[rep_cnt++] = i;
```

```
37     }
38 }
39
40 printf("%d\n", rep_cnt - 1);
41 for (int i = 1; i < rep_cnt; i++) {
42     printf("%d %d\n", reps[i - 1], reps[i]);
43 }
44
45 return 0;
46}
47
```

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t1.txt

4 2

1 2

3 4

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Graph4.exe

1

2 4

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t2.txt

4 2

2 3

2 4

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Graph4.exe

1

1 4

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>

### Question description

You have an undirected graph consisting of  $n$  vertices with weighted edges.

A simple cycle is a cycle of the graph without repeated vertices. Let the *weight* of the cycle be the **XOR** of weights of edges it consists of.

Let's say the graph is *good* if all its *simple* cycles have weight 1. A graph is bad if it's not good.

Initially, the graph is empty. Then  $q$  queries follow. Each query has the next type:

- $u \ v \ x$  — add edge between vertices  $u$  and  $v$  of weight  $x$  if it doesn't make the graph bad.

For each query print, was the edge added or not.

### Constraints

$$3 \leq n \leq 3 \cdot 10^5; 1 \leq q \leq 5 \cdot 10^5$$

$$1 \leq u, v \leq n; u \neq v; 0 \leq x \leq 1$$

### Input

The first line contains two integers  $n$  and  $q$  — the number of vertices and queries.

Next  $q$  lines contain queries — one per line. Each query contains three integers  $u$ ,  $v$  and  $x$  — the vertices of the edge and its weight.

**It's guaranteed that there are no multiple edges in the input.**

### Output

For each query, print YES if the edge was added to the graph, or NO otherwise (both case-insensitive).

Logical Test Cases

Test Case 1

INPUT (STDIN)

9 12  
6 1 0  
1 3 1  
3 6 0  
6 2 0  
6 4 1  
3 4 1  
2 4 0  
2 5 0  
4 5 0  
7 8 1  
8 9 1  
9 7 0

EXPECTED OUTPUT

YES  
YES  
YES  
YES  
YES  
NO  
YES  
YES  
NO  
YES  
YES  
NO

Test Case 2

INPUT (STDIN)

9 6  
1 3 1  
3 6 0  
6 4 1  
2 4 0  
4 5 0  
7 8 1

EXPECTED OUTPUT

YES  
YES  
YES  
YES  
YES  
YES

Mandatory Test Cases

Test Case 1

KEYWORD

int dfs1(int np,int lst)

✓ Complexity Test Cases

**Test Case 1**

CYCLOMATIC COMPLEXITY

**12**

**Test Case 2**

TOKEN COUNT

**1255**

**Test Case 3**

NLOC

**78**

Graph5.c +

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <string.h>
4
5 #define MAXN 300005
6
7 typedef struct Edge {
8     int to;
9     int weight;
10    int id;
11    int in_cycle;
12    struct Edge *nxt;
13    struct Edge *rev;
14} Edge;
15
16 Edge *head[MAXN];
17 int visited[MAXN];
18 int path_nodes[MAXN];
19 Edge *path_edges[MAXN];
20 int path_len = 0;
21 int target_node = 0;
22 int found = 0;
23
24 void add_edge_graph(int u, int v, int w, int id) {
25     Edge *e1 = (Edge*)malloc(sizeof(Edge));
```

```
26 Edge *e2 = (Edge*)malloc(sizeof(Edge));
27
28 e1->to = v;
29 e1->weight = w;
30 e1->id = id;
31 e1->in_cycle = 0;
32 e1->nxt = head[u];
33 e1->rev = e2;
34 head[u] = e1;
35
36 e2->to = u;
37 e2->weight = w;
38 e2->id = id;
39 e2->in_cycle = 0;
40 e2->nxt = head[v];
41 e2->rev = e1;
42 head[v] = e2;
43}
44
45int dfsi(int np, int lst) {
```

```

46     visited[np] = 1;
47     if (np == target_node) {
48         found = 1;
49         return 1;
50     }
51     for (Edge *e = head[np]; e; e = e->nxt) {
52         int v = e->to;
53         if (v != lst && !visited[v]) {
54             path_edges[path_len] = e;
55             path_nodes[path_len] = v;
56             path_len++;
57             if (dfs1(v, np)) return 1;
58             path_len--;
59         }
60     }
61     return 0;
62 }
63
64 int parent_dsu[MAXN];
65 int find_dsu(int i) {

```

```

66     if (parent_dsu[i] == i) return i;
67     return parent_dsu[i] = find_dsu(parent_dsu[i]);
68 }
69
70 int main() {
71     int n, q;
72     if (scanf("%d %d", &n, &q) != 2) return 0;

```

main | CH.SC.U4CSE25225 | Graph5.c

Ln 143, Col 1 Spaces: 4 UTF-8 CRLF C

```

73
74     for (int i = 1; i <= n; i++) {
75         parent_dsu[i] = i;
76         head[i] = NULL;
77     }
78
79     int edge_id_counter = 0;
80
81     for (int k = 0; k < q; k++) {
82         int u, v, x;
83         if (scanf("%d %d %d", &u, &v, &x) != 3) break;
84
85         int ru = find_dsu(u);

```

```

86     int rv = find_dsu(v);
87
88     if (ru != rv) {
89         parent_dsu[ru] = rv;
90         add_edge_graph(u, v, x, ++edge_id_counter);
91         printf("YES\n");
92     } else {
93         path_len = 0;
94         target_node = v;
95         found = 0;
96         for (int i = 1; i <= n; i++) visited[i] = 0;
97
98         path_nodes[0] = u;
99         dfsi(u, 0);
100
101         int has_cycle_edge = 0;
102         int xor_sum = 0;
103         for (int i = 0; i < path_len; i++) {
104             if (path_edges[i]->in_cycle) {
105                 has_cycle_edge = 1;

```

```

106         break;
107     }
108     xor_sum ^= path_edges[i]->weight;
109 }
110
111 if (!has_cycle_edge && (xor_sum ^ x) == 1) {
112     for (int i = 0; i < path_len; i++) {
113         path_edges[i]->in_cycle = 1;
114         path_edges[i]->rev->in_cycle = 1;
115     }
116     Edge *new_e1 = (Edge*)malloc(sizeof(Edge));
117     Edge *new_e2 = (Edge*)malloc(sizeof(Edge));
118     new_e1->to = v;
119     new_e1->weight = x;
120     new_e1->id = ++edge_id_counter;
121     new_e1->in_cycle = 1;
122     new_e1->nxt = head[u];
123     new_e1->rev = new_e2;
124     head[u] = new_e1;

```

```
125
126         new_e2->to = u;
127         new_e2->weight = x;
128         new_e2->id = edge_id_counter;
129         new_e2->in_cycle = 1;
130         new_e2->nxt = head[v];
131         new_e2->rev = new_e1;
132         head[v] = new_e2;
133
134         printf("YES\n");
135     } else {
136         printf("NO\n");
137     }
138 }
139 }
140
141 return 0;
142 }
143
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t1.txt
```

```
9 12
```

```
6 1 0
```

```
1 3 1
```

```
3 6 0
```

```
6 2 0
```

```
6 4 1
```

```
3 4 1
```

```
2 4 0
```

```
2 5 0
```

```
4 5 0
```

```
7 8 1
```

```
8 9 1
```

```
9 7 0
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Graph5.exe
```

```
YES
```

```
YES
```

```
YES
```

```
YES
```

```
YES
```

```
NO
```

```
YES
```

```
YES
```

```
NO
```

```
YES
```

```
YES
```

```
NO
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t2.txt
```

```
9 6
1 3 1
3 6 0
6 4 1
2 4 0
4 5 0
7 8 1
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Graph5.exe
```

YES

YES

YES

YES

YES

YES

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>

### Question description

There are  $n$  boys and  $m$  girls in a school. Next week a school dance will be organized. A dancing pair consists of a boy and a girl, and there are  $k$  potential pairs.

Your task is to find out the maximum number of dance pairs and show how this number can be achieved.

### Constraints

- $1 \leq n, m \leq 500$
- $1 \leq k \leq 1000$
- $1 \leq a \leq n$
- $1 \leq b \leq m$

### Input

The first input line has three integers  $n$ ,  $m$  and  $k$ : the number of boys, girls, and potential pairs. The boys are numbered  $1, 2, \dots, n$ , and the girls are numbered  $1, 2, \dots, m$ .

After this, there are  $k$  lines describing the potential pairs. Each line has two integers  $a$  and  $b$ : boy  $a$  and girl  $b$  are willing to dance together.

### Output

First print one integer  $rr$ : the maximum number of dance pairs. After this, print lines describing the pairs. You can print any valid solution.

▼ Logical Test Cases

Test Case 1

INPUT (STDIN)

3 2 4  
1 1  
1 2  
2 1  
3 1

EXPECTED OUTPUT

2  
1 2  
2 1

Test Case 2

INPUT (STDIN)

3 2 4  
1 1  
1 2  
2 3  
3 2

EXPECTED OUTPUT

3  
1 1  
2 3  
3 2

▼ Mandatory Test Cases

Test Case 1

KEYWORD

void link(int u,int v)

Test Case 2

KEYWORD

int bfs(int n)

▼ Complexity Test Cases

Test Case 1

CYCLOMATIC COMPLEXITY

2

Test Case 2

TOKEN COUNT

600

Test Case 3

NLOC

84

Graph6.c +

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <string.h>
4
5 #define MAXN 505
6 #define MAXM 1005
7
8 int head[MAXN], to[MAXM], nxt[MAXM], edge_cnt;
9 int match[MAXN], rmatch[MAXN];
10 int dist[MAXN];
11 int visited[MAXN];
12
13 void link(int u, int v) {
14     to[++edge_cnt] = v;
15     nxt[edge_cnt] = head[u];
16     head[u] = edge_cnt;
17 }
18
19 int bfs(int n) {
20     int q[MAXN], front = 0, rear = 0;
21     for (int i = 1; i <= n; i++) {
22         if (match[i] == 0) {
23             dist[i] = 0;
24             q[rear++] = i;
25         } else {
```

```

26         dist[i] = -1;
27     }
28 }
29 int reached_free = 0;
30 while (front < rear) {
31     int u = q[front++];
32     for (int e = head[u]; e; e = nxt[e]) {
33         int v = to[e];
34         int next_u = rmatch[v];
35         if (next_u == 0) {
36             reached_free = 1;
37         } else if (dist[next_u] == -1) {
38             dist[next_u] = dist[u] + 1;
39             q[rear++] = next_u;
40         }
41     }
42 }

```

```
43     return reached_free;
44 }
45
```

main | CH.SC.U4CSE25225 | Graph6.c

Ln 90, Col 1 Spaces: 4 UTF-8 CRLF C

```
47     for (int e = head[u]; e; e = nxt[e]) {
48         int v = to[e];
49         int next_u = rmatch[v];
50         if (next_u == 0 || (dist[next_u] == dist[u] + 1 && dfs(next_u))) {
51             match[u] = v;
52             rmatch[v] = u;
53             return 1;
54         }
55     }
56     dist[u] = -1;
57     return 0;
58 }
```

```

59
60int main() {
61    int n, m, k;
62    if (scanf("%d %d %d", &n, &m, &k) != 3) return 0;
63
64    for (int i = 0; i < k; i++) {
65        int u, v;
66        scanf("%d %d", &u, &v);
67        link(u, v);
68    }
69
70    int matching = 0;
71    while (bfs(n)) {
72        for (int i = 1; i <= n; i++) {
73            if (match[i] == 0) {
74                if (dfs(i)) {

```

```
75         matching++;
76     }
77 }
78 }
79 }
80
81 printf("%d\n", matching);
82 for (int i = 1; i <= n; i++) {
83     if (match[i] != 0) {
84         printf("%d %d\n", i, match[i]);
85     }
86 }
87
88 return 0;
89}
90
```

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t1.txt

3 2 4

1 1

1 2

2 1

3 1

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Graph6.exe

2

1 2

2 1

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t2.txt

3 2 4

1 1

1 2

2 3

3 2

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Graph6.exe

3

1 1

2 3

3 2

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>

**Question description**

There are  $n$  cities and initially no roads between them. However, every day a new road will be constructed, and there will be a total of  $m$  roads.

A component is a group of cities where there is a route between any two cities using the roads. After each day, your task is to find the number of components and the size of the largest component.

**Input**

The first input line has two integers  $n$  and  $m$ : the number of cities and roads. The cities are numbered  $1, 2, \dots, n$ .

Then, there are  $m$  lines describing the new roads. Each line has two integers  $a$  and  $b$ : a new road is constructed between cities  $a$  and  $b$ .

You may assume that every road will be constructed between two different cities.

**Output**

Print  $m$  lines: the required information after each day.

**Constraints**

- $1 \leq n \leq 10^5$
- $1 \leq m \leq 2 \cdot 10^5$
- $1 \leq a, b \leq n$

| Test Case 1                                   |
|-----------------------------------------------|
| INPUT (STDIN)                                 |
| 5 6<br>1 2<br>2 3<br>3 1<br>3 4<br>4 5<br>5 4 |
| EXPECTED OUTPUT                               |
| 4 2<br>3 3<br>3 3<br>2 4<br>1 5<br>1 5        |

| Test Case 2                            |
|----------------------------------------|
| INPUT (STDIN)                          |
| 5 5<br>1 3<br>2 4<br>3 1<br>3 2<br>5 4 |
| EXPECTED OUTPUT                        |
| 4 2<br>3 2<br>3 2<br>2 4<br>1 5        |

▼ Mandatory Test Cases

| Test Case 1                        |
|------------------------------------|
| KEYWORD                            |
| <code>int join(int i,int j)</code> |

▼ Complexity Test Cases

| Test Case 1           |
|-----------------------|
| CYCLOMATIC COMPLEXITY |
| 2                     |

| Test Case 2 |
|-------------|
| TOKEN COUNT |
| 300         |

| Test Case 3 |
|-------------|
| NLOC        |
| 40          |

```
Graph7.c +
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 #define MAXN 200005
5
6 int parent[MAXN];
7 int sz[MAXN];
8 int num_components;
9 int max_comp_size;
10
11 int find(int i) {
12     if (parent[i] == i)
13         return i;
14     return parent[i] = find(parent[i]);
15 }
16
17 int join(int i, int j) {
18     int root_i = find(i);
19     int root_j = find(j);
20     if (root_i != root_j) {
21         if (sz[root_i] < sz[root_j]) {
22             int temp = root_i;
23             root_i = root_j;
24             root_j = temp;
25         }
26     }
27 }
```

```
26     parent[root_j] = root_i;
27     sz[root_i] += sz[root_j];
28     if (sz[root_i] > max_comp_size) {
29         max_comp_size = sz[root_i];
```

main | CH.SC.U4CSE25225 | Graph7.c

Ln 57, Col 1 Spaces: 4 UTF-8 CRLF C

```
30     }
31     num_components--;
32     return 1;
33 }
34 return 0;
35}
36
```

```
37int main() {
38    int n, m;
39    if (scanf("%d %d", &n, &m) != 2) return 0;
40
41    for (int i = 1; i <= n; i++) {
42        parent[i] = i;
43        sz[i] = 1;
44    }
45    num_components = n;
46    max_comp_size = 1;
47
```

```
48     for (int i = 0; i < m; i++) {
49         int u, v;
50         scanf("%d %d", &u, &v);
51         join(u, v);
52         printf("%d %d\n", num_components, max_comp_size);
53     }
54
55     return 0;
56 }
57
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t1.txt
```

```
5 6
```

```
1 2
```

```
2 3
```

```
3 1
```

```
3 4
```

```
4 5
```

```
5 4
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Graph7.exe
```

```
4 2
```

```
3 3
```

```
3 3
```

```
2 4
```

```
1 5
```

```
1 5
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t2.txt
```

```
5 5
```

```
1 3
```

```
2 4
```

```
3 1
```

```
3 2
```

```
5 4
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Graph7.exe
```

```
4 2
```

```
3 2
```

```
3 2
```

```
2 4
```

```
1 5
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>
```



|    |         |       |                      |         |              |
|----|---------|-------|----------------------|---------|--------------|
| DS | Session | Graph | Question Information | Level 1 | Challenge 88 |
|----|---------|-------|----------------------|---------|--------------|

Question description

You are given a tree consisting of  $n$  nodes. A *matching* is a set of edges where each node is an endpoint of at most one edge. What is the maximum number of edges in a matching?

Input Format

The first input line contains an integer  $n$ : the number of nodes. The nodes are numbered  $1, 2, \dots, n$ .

Then there are  $n-1$  lines describing the edges. Each line contains two integers  $a$  and  $b$ : there is an edge between nodes  $a$  and  $b$ .

Output Format

Print one integer: the maximum number of pairs.

Constraints

- $1 \leq n \leq 2 \cdot 10^5$
- $1 \leq a, b \leq n$

Example

Input:

5  
1 2  
1 3  
3 4  
3 5

Output:

2

Explanation: One possible matching is (1,2) and (3,4)

Logical Test Cases

Test Case 1

INPUT (STDIN)

5  
1 2  
1 3  
3 4  
3 5

EXPECTED OUTPUT

2

Test Case 2

INPUT (STDIN)

5  
1 1  
1 2  
2 3  
3 4

EXPECTED OUTPUT

6

Mandatory Test Cases

Test Case 1

KEYWORD

void dfs(int p,int i)

Test Case 2

KEYWORD

void link(int i,int j)

Complexity Test Cases

Test Case 1

CYCLOMATIC COMPLEXITY

9

Test Case 2

TOKEN COUNT

350

Test Case 3

NLOC

50

Graph8.c +

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 #define MAXN 200005
5
6 int head[MAXN], to[MAXN * 2], nxt[MAXN * 2], edge_cnt;
7 int matched[MAXN];
8 int ans = 0;
9
10 void link(int i, int j) {
11     to[++edge_cnt] = j;
12     nxt[edge_cnt] = head[i];
13     head[i] = edge_cnt;
14 }
15
16 void dfs(int p, int i) {
17     for (int e = head[i]; e; e = nxt[e]) {
18         int v = to[e];
19         if (v != p) {
20             dfs(i, v);
21         }
22     }
23     if (p != 0 && !matched[i] && !matched[p]) {
24         matched[i] = 1;
25         matched[p] = 1;
```

main | CH.SC.U4CSE25225 | Graph8.c

Ln 43, Col 1 Spaces: 4 UTF-8 CRLF C

```
26     ans++;
27 }
28}
29
30int main() {
31     int n;
32     if (scanf("%d", &n) != 1) return 0;
33     for (int i = 0; i < n - 1; i++) {
34         int u, v;
```

```
35     scanf("%d %d", &u, &v);
36     link(u, v);
37     link(v, u);
38 }
39 dfs(0, 1);
40 printf("%d\n", ans);
41 return 0;
42}
43
```

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t1.txt

5

1 2

1 3

3 4

3 5

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Graph8.exe

2

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t2.txt

5

1 1

1 2

2 3

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Graph8.exe

2

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>



INPUT (STDIN)

6 7

1 2

1 3

2 6

3 4

3 5

4 6

5 6

EXPECTED OUTPUT

2

4

1 3 5 6

3

1 2 6

INPUT (STDIN)

6 7

1 4

3 5

4 6

5 6

2 2

1 3

2 6

EXPECTED OUTPUT

2

4

1 3 5 6

3

1 4 6

▼ Mandatory Test Cases

Test Case 1

KEYWORD

int bfs(int n,int s,int t)

▼ Complexity Test Cases

Test Case 1

CYCLOMATIC COMPLEXITY

7

Test Case 2

TOKEN COUNT

920

Test Case 3

NLOC

110

Graph9.c +

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <string.h>
4
5 #define MAXN 505
6 #define MAXM 2005
7
8 int head[MAXN], to[MAXM * 2], cap[MAXM * 2], flow[MAXM * 2], nxt[MAXM * 2], edge_cnt;
9 int parent_edge[MAXN];
10 int parent_node[MAXN];
11
12 void add_edge(int u, int v) {
13     to[edge_cnt] = v;
14     cap[edge_cnt] = 1;
15     flow[edge_cnt] = 0;
16     nxt[edge_cnt] = head[u];
17     head[u] = edge_cnt++;
18
19     to[edge_cnt] = u;
20     cap[edge_cnt] = 0;
21     flow[edge_cnt] = 0;
22     nxt[edge_cnt] = head[v];
23     head[v] = edge_cnt++;
24 }
25
```

```

26 int bfs(int n, int s, int t) {
27     int q[MAXN], front = 0, rear = 0;
28     int visited[MAXN];
29     memset(visited, 0, sizeof(visited));
30     memset(parent_edge, -1, sizeof(parent_edge));
31
32     q[rear++] = s;
33     visited[s] = 1;
34
35     while (front < rear) {
36         int u = q[front++];
37         if (u == t) return 1;
38
39         for (int e = head[u]; e != -1; e = nxt[e]) {
40             int v = to[e];
41             if (!visited[v] && cap[e] - flow[e] > 0) {
42                 visited[v] = 1;

```

```

43         parent_edge[v] = e;
44         parent_node[v] = u;
45         q[rear++] = v;
46     }
47 }
48 }
49 return 0;
50}

```

```
51
```

```
52int max_flow(int n, int s, int t) {
```

```
53     int total_flow = 0;
```

```
54     while (bfs(n, s, t)) {
```

```
55         for (int v = t; v != s; v = parent_node[v]) {
```

```
56             int e = parent_edge[v];
```

```
57             flow[e] += 1;
```

```
58             flow[e ^ 1] -= 1;
```

```
59         }
```

```
60     total_flow++;
61 }
62 return total_flow;
63}
64
65int path_nodes[MAXN];
66
67int main() {
68     int n, m;
69     if (scanf("%d %d", &n, &m) != 2) return 0;
70
71     memset(head, -1, sizeof(head));
72     edge_cnt = 0;
73
74     for (int i = 0; i < m; i++) {
75         int u, v;
```

```
76     scanf("%d %d", &u, &v);
77     add_edge(u, v);
78 }
79
80 int k = max_flow(n, 1, n);
81 printf("%d\n", k);
82
83 for (int i = 0; i < k; i++) {
84     int curr = 1;
85     int path_len = 0;
86     path_nodes[path_len++] = curr;
87
88     while (curr != n) {
89         for (int e = head[curr]; e != -1; e = nxt[e]) {
90             if (flow[e] > 0) {
91                 flow[e] = 0;
```

```

92         curr = to[e];
93         path_nodes[path_len++] = curr;
94         break;
95     }
96 }
97 }
98
99 printf("%d\n", path_len);
100 for (int j = 0; j < path_len; j++) {
101     printf("%d%c", path_nodes[j], j == path_len - 1 ? '\n' : ' ');
102 }
103 }
104
105 return 0;
106 }
107

```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t1.txt
```

```
6 7
```

```
1 2
```

```
1 3
```

```
2 6
```

```
3 4
```

```
3 5
```

```
4 6
```

```
5 6
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Graph9.exe
```

```
2
```

```
4
```

```
1 3 5 6
```

```
3
```

```
1 2 6
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t2.txt
```

```
6 7
```

```
1 4
```

```
3 5
```

```
4 6
```

```
5 6
```

```
2 2
```

```
1 3
```

```
2 6
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Graph9.exe
```

```
2
```

```
4
```

```
1 3 5 6
```

3

1 4 6

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>
```

### Question description

Kaaleppi has just robbed a bank and is now heading to the harbor. However, the police wants to stop him by closing some streets of the city.

What is the minimum number of streets that should be closed so that there is no route between the bank and the harbor?

### Constraints

- $2 \leq n \leq 500$
- $1 \leq m \leq 1000$
- $1 \leq a, b \leq n$

### Input

The first input line has two integers  $n$  and  $m$ : the number of crossings and streets. The crossings are numbered  $1, 2, \dots, n$ . The bank is located at crossing 1, and the harbor is located at crossing  $n$ .

After this, there are  $m$  lines that describe the streets. Each line has two integers  $a$  and  $b$ : there is a street between crossings  $a$  and  $b$ . All streets are two-way streets, and there is at most one street between two crossings.

### Output

First print an integer  $k$ : the minimum number of streets that should be closed. After this, print  $k$  lines describing the streets. You can print any valid solution.

▼ Logical Test Cases

Test Case 1

INPUT (STDIN)  
  
4 5  
1 2  
1 3  
2 3  
3 4  
1 4  
  
EXPECTED OUTPUT  
  
2  
1 4  
3 4

Test Case 2

INPUT (STDIN)  
  
4 5  
1 1  
2 2  
2 3  
3 4  
1 4  
  
EXPECTED OUTPUT  
  
1  
1 4

▼ Mandatory Test Cases

Test Case 1

KEYWORD  
  
void link(int i,int h)

Test Case 2

KEYWORD  
  
int bfs(int n,int s,int t)

▼ Complexity Test Cases

Test Case 1

CYCLOMATIC COMPLEXITY  
  
1

Test Case 2

TOKEN COUNT  
  
735

Test Case 3

NLOC  
  
93

Graph10.c +

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <string.h>
4
5 #define MAXN 505
6 #define MAXM 1005
7
8 int head[MAXN], to[MAXM * 4], cap[MAXM * 4], flow[MAXM * 4], nxt[MAXM * 4], edge_cnt;
9 int orig_u[MAXM], orig_v[MAXM];
10 int parent_edge[MAXN], parent_node[MAXN];
11 int visited[MAXN];
12
13 void link(int i, int h) {
14     to[edge_cnt] = h;
15     cap[edge_cnt] = 1;
16     flow[edge_cnt] = 0;
17     nxt[edge_cnt] = head[i];
18     head[i] = edge_cnt++;
19
20     to[edge_cnt] = i;
21     cap[edge_cnt] = 1;
22     flow[edge_cnt] = 0;
23     nxt[edge_cnt] = head[h];
24     head[h] = edge_cnt++;
25 }
```

```

26
27int bfs(int n, int s, int t) {
28    int q[MAXN], front = 0, rear = 0;
29    memset(visited, 0, sizeof(visited));
30    memset(parent_edge, -1, sizeof(parent_edge));
31
32    q[rear++] = s;
33    visited[s] = 1;
34
35    while (front < rear) {
36        int u = q[front++];
37        if (u == t) return 1;
38
39        for (int e = head[u]; e != -1; e = nxt[e]) {
40            int v = to[e];
41            if (!visited[v] && cap[e] - flow[e] > 0) {
42                visited[v] = 1;
43                parent_edge[v] = e;

```

```

44         parent_node[v] = u;
45         q[rear++] = v;
46     }
47 }
48 }
49 return 0;
50}
51
52int max_flow(int n, int s, int t) {
53     int total_flow = 0;
54     while (bfs(n, s, t)) {
55         for (int v = t; v != s; v = parent_node[v]) {
56             int e = parent_edge[v];
57             flow[e] += 1;
58             flow[e ^ 1] -= 1;
59         }
60         total_flow++;

```

```
61     }
62     return total_flow;
63 }
64
65 void mark_reachable(int s) {
66     int q[MAXN], front = 0, rear = 0;
67     memset(visited, 0, sizeof(visited));
68
69     q[rear++] = s;
70     visited[s] = 1;
71
72     while (front < rear) {
73         int u = q[front++];
74         for (int e = head[u]; e != -1; e = nxt[e]) {
75             int v = to[e];
76             if (!visited[v] && cap[e] - flow[e] > 0) {
77                 visited[v] = 1;
```

```

78         q[rear++] = v;
79     }
80 }
81 }
82}
83
84int main() {
85     int n, m;
86     if (scanf("%d %d", &n, &m) != 2) return 0;
87
88     memset(head, -1, sizeof(head));
89     edge_cnt = 0;
90
91     for (int i = 0; i < m; i++) {
92         scanf("%d %d", &orig_u[i], &orig_v[i]);
93         link(orig_u[i], orig_v[i]);
94     }

```

```
95
96     int k = max_flow(n, 1, n);
97     printf("%d\n", k);
98
99     mark_reachable(1);
100
101     for (int i = 0; i < m; i++) {
102         int u = orig_u[i];
103         int v = orig_v[i];
104         if ((visited[u] && !visited[v]) || (visited[v] && !visited[u])) {
105             printf("%d %d\n", u, v);
106         }
107     }
108
109     return 0;
110 }
111
```

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t1.txt

4 5

1 2

1 3

2 3

3 4

1 4

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Graph10.exe

2

3 4

1 4

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t2.txt

4 5

1 1

2 2

2 3

3 4

1 4

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Graph10.exe

1

1 4

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>